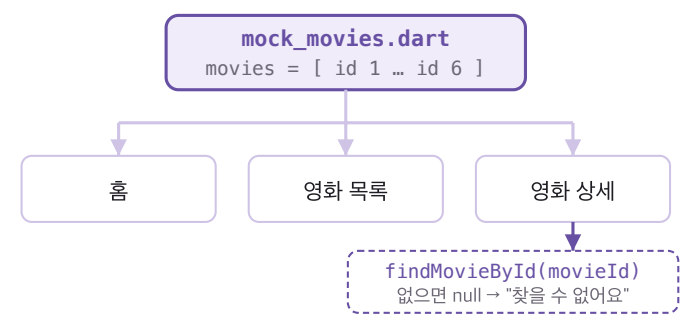


Mock Movie 데이터 · ListView · ListView.separated · GridView

1. Mock 데이터는 한곳에 — 세 화면이 같은 것을 읽는다



세 화면이 각자 하드코딩하면 같은 영화가 화면마다 다르게 보인다.

```
class Movie {
  const Movie({ required this.id, required
    this.title,
    required this.genre, required this.year,
    required this.posterAsset });
  final int id; // 상세 Route의 Path
  Parameter
  ""
}
```

왜 Movie? (nullable)로 돌려주나

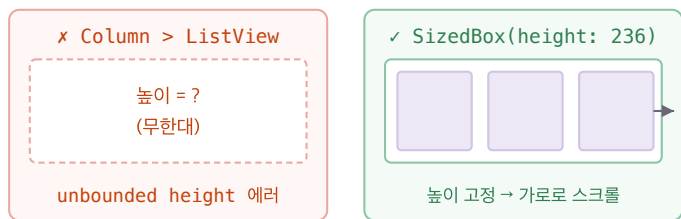
/movies/999 처럼 없는 ID로 들어올 수 있다. null이면 "없음" 화면을 그리면 되지만, !로 강제하면 앱이 죽는다.

2. 세 위젯의 차이 — 셋 다 itemCount + itemBuilder, GridView만 gridDelegate가 더 붙는다



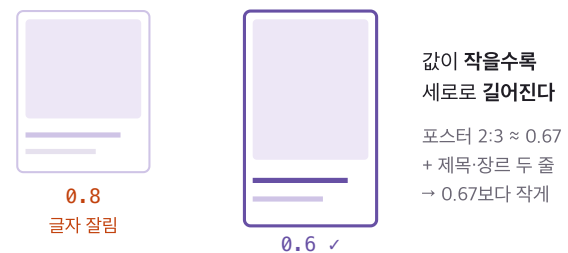
Widget	쓰는 경우	내 화면
ListView.builder	한 방향으로 나열	—
ListView.separated	항목 사이에 간격·구분선	홈 가로 줄(12px), 장르 Chip 줄(8px)
GridView.builder	한 줄에 2개 이상	영화 목록 2열

3. 가로 목록은 높이를 못 정한다



세로 목록은 부모가 높이를 주지만 가로 목록은 아무도 높이를 안 준다. 카드 너비도 직접 정한다(width: 136).

4. childAspectRatio = 가로 ÷ 세로



비율만 정하면 실제 크기는 화면 너비에 따라 정해진다. 글자가 잘리면 값을 더 줄인다.

커넥션 풀 (Connection Pool)

1. 문제 — 연결을 맺는 시간이 쿼리보다 길다

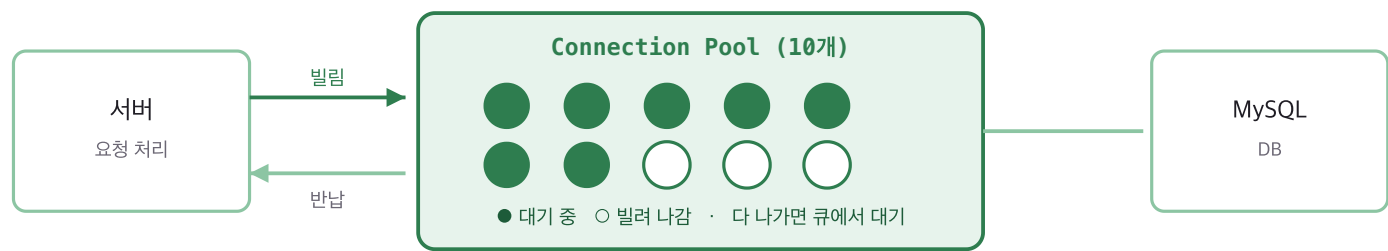
풀 없음 — 요청마다 새로 연결



커넥션 풀 — 미리 뚫어두고 빌려 씀



2. 풀은 어떻게 동작하나



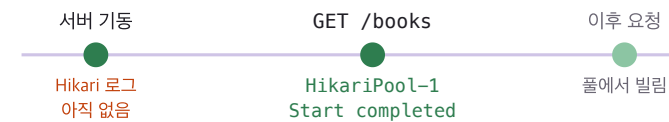
connectionLimit 10 · waitForConnections true · queueLimit 0

서버 시작 때 미리 연결해 두고, 요청마다 빌려주고 돌려받는 바구니. 연결 생성 비용이 요청 처리 경로에서 사라진다.

3. Spring은 자동, NestJS는 직접

구분	Spring Boot	NestJS
풀 구현	HikariCP 내장	mysql2 createPool
설정	application.yml 만 쓰면 자동 생성	Provider로 직접 조립
쓸 때	JdbcTemplate 주입	@Inject(...)로 주입

4. 풀은 첫 쿼리 때 만들어진다



기동 로그에 Hikari가 없다고 연결 실패가 아니다 — lazy 초기화.

5. 주의할 점 두 가지

커넥션 누수

빌린 커넥션을 반납하지 않으면 풀이 말라 서버 전체가 멈춘다. JdbcTemplate 이 빌리고 반납까지 하므로, Repository에서 커넥션을 직접 열고 닫지 않는 것이 최선의 방어.

풀은 클수록 좋은가 → 아니다

커넥션 하나당 DB 쪽에도 스레드·메모리가 붙는다. 앱 서버가 여러 대면 DB의 max_connections 를 넘겨 오히려 죽는다. 기본 10에서 시작해 측정 후 조정.

```
# application.yml - Spring은 이게 전부
spring:
  datasource:
    url:
    jdbc:mysql://localhost:3306/book_rental
    username: ${DB_USER:root}
    password: ${DB_PASSWORD:}
```